

System Flow

DAC Executive Dashboard - architecture and end-to-end data flows

Document	System Flow
Application	Con Edison Disadvantaged Communities (DAC) Executive Dashboard
Environment	Clara Fortuna Dev (Developer environment with Dataverse) - TEST build
Status	Current as of June 24, 2026. Production move is a separate, future phase.

1. Purpose And Scope

This document describes how the **DAC Executive Dashboard** is put together and how data moves through it - from the moment a user opens the app to the point where an edit is stored and audited. It covers the running application, its storage layer, the Microsoft Dataverse backend, and the development-and-deployment pipeline that keeps the running copy in sync with source control.

The dashboard presents Con Edison's annual Disadvantaged Communities (DAC) report data - ten regulatory sections (A through J) of tables, KPIs, charts, and a service-territory map - and let authorized users ingest and edit the underlying numbers in place, with every change recorded.

Environment note: The system currently runs in the Clara Fortuna Dev environment as a TEST build (tables are named "... Test"). The mechanism is identical to what production will use; the production cutover is gated on separate technical and cybersecurity approvals and is out of scope for this document.

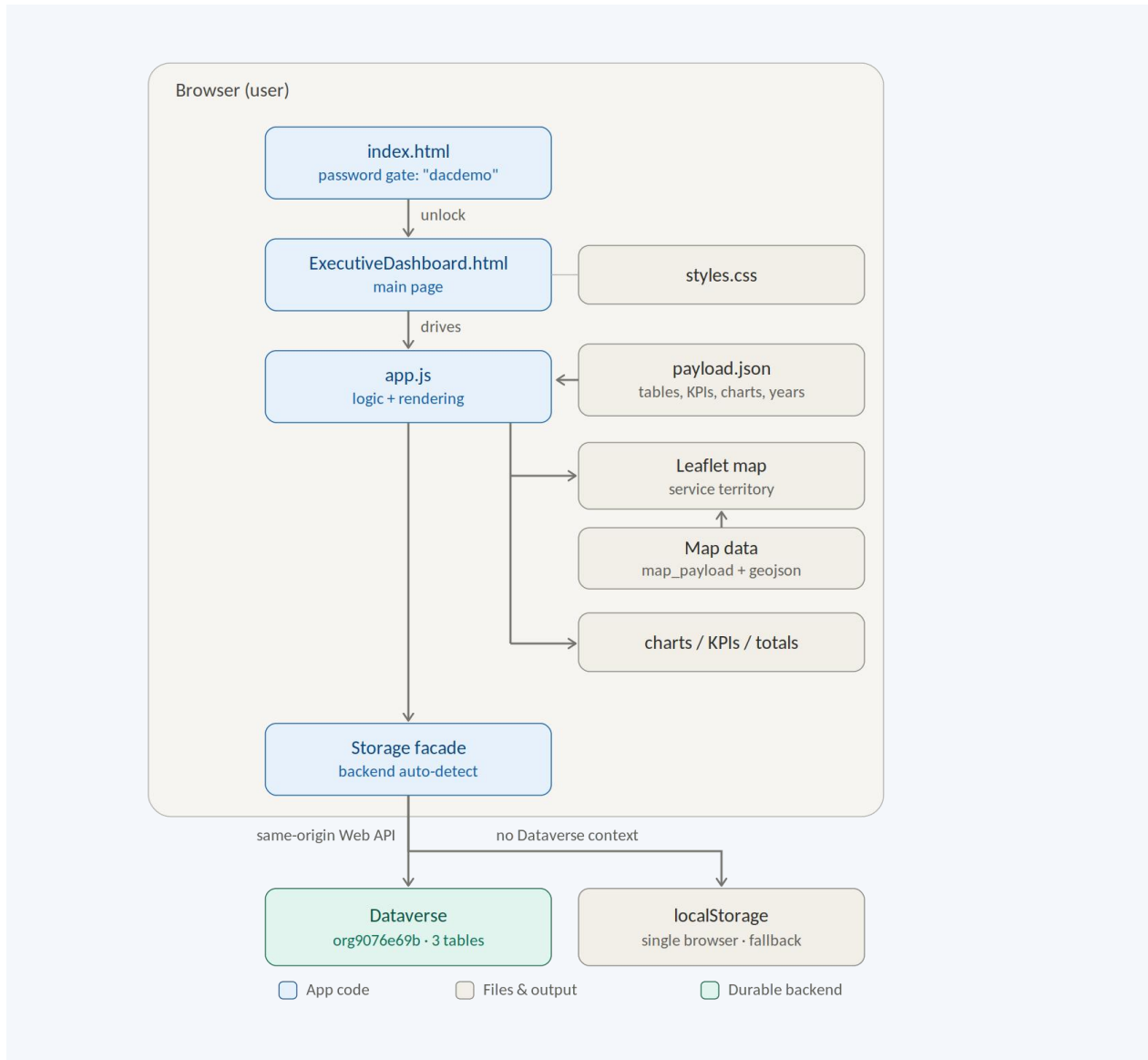
2. Architecture

The system has three layers that are intentionally decoupled: a **presentation layer** (the dashboard UI), a **storage facade** (a thin layer that abstracts where data live), and a **persistence layer** (Microsoft Dataverse, with a browser-local fallback). The UI never talks to Dataverse directly - it always goes through the facade, which means the backend can be swapped without touching the UI.

Layer	What it is	Responsibility
Presentation	ExecutiveDashboard.html + app.js + styles.css + payload.json	Renders the password gate, sections, KPIs, charts, the Leaflet map, and the in-grid ingestion editor.
Storage facade	The "Storage" module inside app.js	Detects the backend at startup, then exposes a single read/write interface to the UI (load, save, add/remove year, change-history write).
Persistence	Dataverse tables (or localStorage fallback)	Durably stores each table's grid, the reporting years, and the change-history audit trail.
Hosting shell	Model-driven Power Apps app + Dataverse web resources	Serves the dashboard files from the org domain and guarantees the Dataverse session is available.

2.1 Component Map

The diagram below shows the components and the direction of data flow between them.



3. Runtime Data Flows

Four flows describe how the system behaves at runtime: startup and backend detection, reading data into the dashboard, the ingestion / edit round-trip, and the audit trail.

3.1 Startup And Backend Detection

When the page loads, the storage facade decides where it is running before any data is read. This is the single most important branch in the system, because it determines whether changes are durable and shared (Dataverse) or local and temporary (localStorage).

1. The browser loads `index.html` (the password gate). The user enters the access password (`dacdemo`) and is forwarded to `ExecutiveDashboard.html`.
2. On load, `app.js` calls `Storage.init()`.
3. The facade checks whether it is being served from the Dataverse org domain. If so, it uses `window.location.origin` as the Dataverse base URL and authenticates with the same-origin session cookies - no helper script required.
4. If there is no Dataverse context (for example when served from `localhost`), it falls back to `localStorage`.
5. The chosen backend is logged to the browser console so it can be verified at a glance.

Console line	Meaning
<code>[Storage] backend = Dataverse (https://org9076e69b.crm.dynamics.com)</code>	Running inside Power Apps / served from the org domain. Edits are durable and shared.
<code>[Storage] backend = localStorage (no Dataverse context)</code>	Running locally or outside the org. Edits stay in that one browser only (expected for local preview).

3.2 Reading Data Into The Dashboard

Once the backend is known, the dashboard assembles each view from two distinct sources: the structure comes from a static file, and the live numbers come from the backend.

6. **Load structure.** `app.js` reads `payload.json` for definitions - which tables and sections exist, their column headers, the charts and KPIs to draw, and the available reporting years.
7. **Load values.** For each table-year, the facade fetches the matching record from Dataverse and parses its stored grid (the JSON in the `cr2bf_rows` field) into rows.
8. **Render.** The dashboard fills the grids, computes totals and sums, draws the charts, and renders the KPIs. The Leaflet map is drawn from `map_payload.json` and `service_territories.geojson`.
9. **Empty state.** A table-year with no record shows a clear empty state rather than stale data.

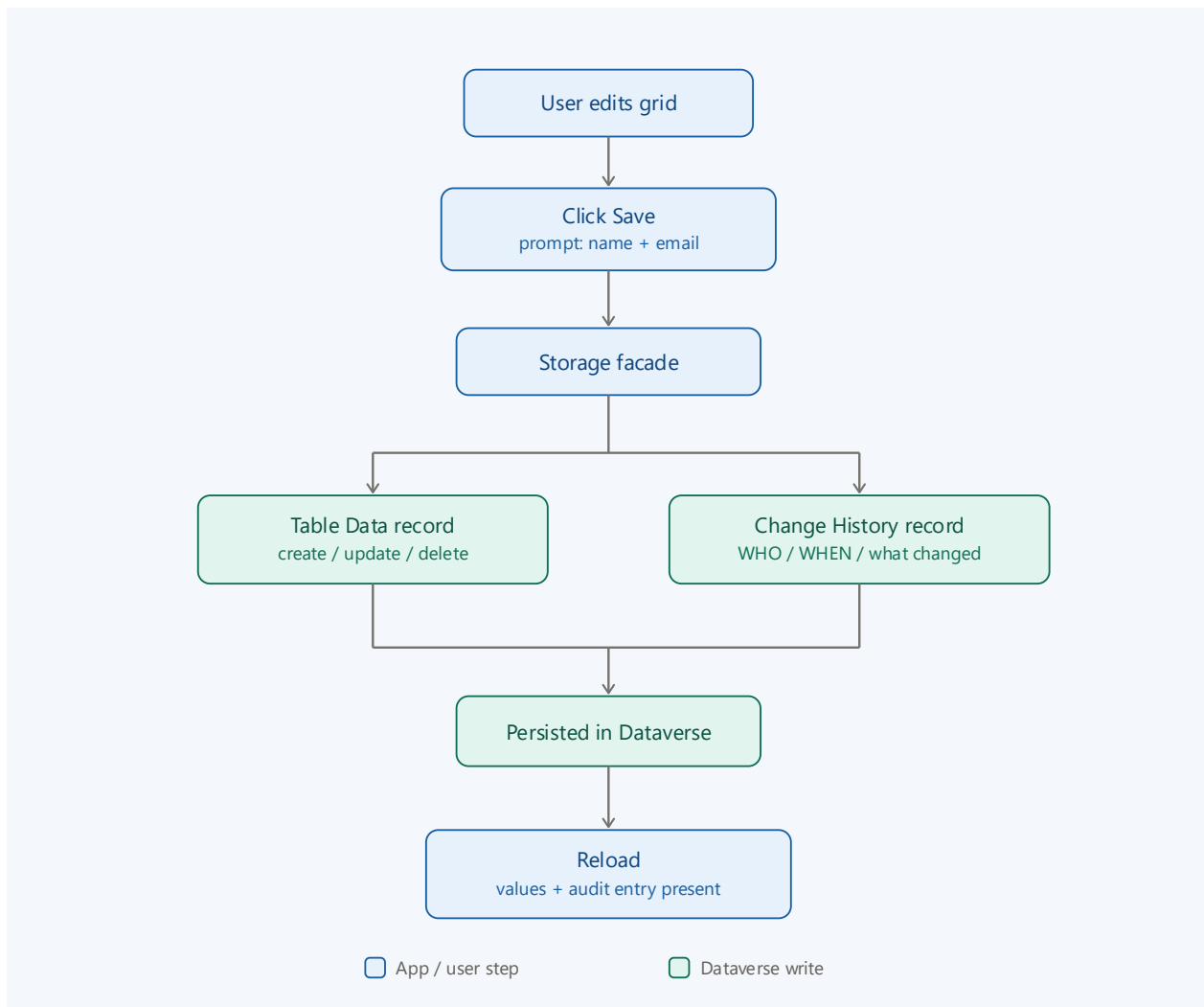
Key separation: `payload.json` holds definitions only; it does not hold editable numbers. The numbers live in Dataverse. This is why a structure change and a data change are handled differently (see the development pipeline in section 4).

3.3 The Ingestion / Edit Round-Trip

The dashboard has an in-UI editor (no separate Power Apps forms). Create, edit, and delete all flow through the same path and all land in Dataverse.

10. **Edit in the grid.** The user edits a cell, adds or removes a row, or adds/removes a reporting year directly in the ingestion area.
11. **Save with identity.** On Save, the app prompts for the user's name and email - this identifies who made the change.
12. **Write the value.** The facade writes the updated grid to the [DAC Ingest Test - Table Data](#) table (creating, updating, or deleting the record as appropriate). Adding a year also writes to the [Reporting Year](#) table.
13. **Write the audit entry.** In the same operation, a row is written to the [Change History](#) table capturing WHO, WHEN, and what changed.
14. **Persist and confirm.** On reload or hard refresh, the change is still present and the console confirms `backend = Dataverse`.

The flow, end to end:



3.4 Audit Trail

Every create, edit, and delete - whether made in the grid editor or, in future, via CSV ingestion - produces a matching entry in the [Change History](#) table. Each entry records the user (name and email), the timestamp, and a description of what changed, giving the report a complete WHO/WHEN audit record.

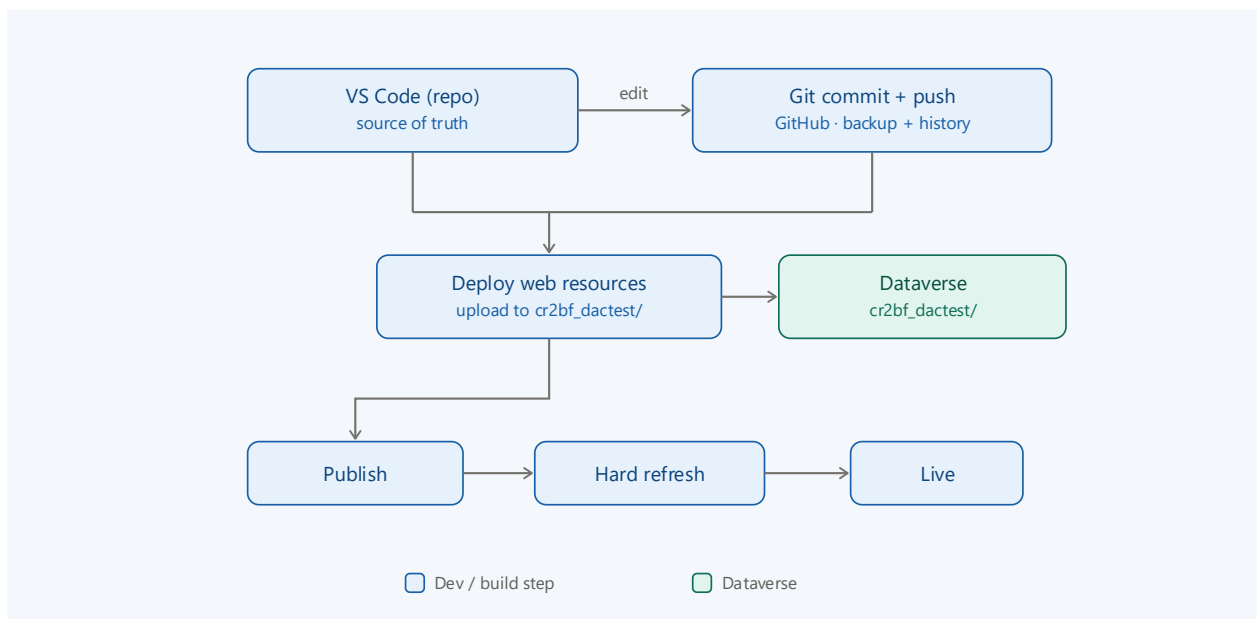
4. Development And Deployment Pipeline

The running copy in Power Apps is never the place to make changes. The **source of truth** is the GitHub repository; the running copy is produced by deploying from it. Editing both independently makes them drift apart.

4.1 The Pipeline

15. **Edit in the repo.** Open the [Coned/CLCPA/ExecutiveDashboard_Test/](#) folder and edit the file(s) you need.
16. **Preview locally (optional).** Serve the folder with a local web server. Locally the app uses the localStorage fallback (`backend = localStorage`), which is fine for checking look and layout but will not show live Dataverse data.
17. **Commit and push to GitHub.** This keeps version history and a backup.
18. **Deploy to the app.** Re-upload the changed file(s) into Dataverse as web resources under [cr2bf_dactest/](#) and Publish - manually in Power Apps (Solutions → CLCPA DAC Dashboard → Web resources → upload → Save → Publish).
19. **Verify.** Open the app and do a hard refresh (Ctrl+F5) or use an incognito window - Power Apps caches web resources aggressively, so a normal refresh often shows the old version.

Visualized:



4.2 Structure Changes Vs. Data Changes

Editing `payload.json` changes **definitions** (tables, headers, charts) - not the live numbers. If a structural change also affects the baseline data, Dataverse must be re-synced afterward (re-run the data loader) so the tables match the new structure.

5. Reference

5.1 Environment And Endpoints

Item	Value
Environment	Clara Fortuna Dev (Developer environment with Dataverse)
Org URL	https://org9076e69b.crm.dynamics.com
Publisher prefix	cr2bf_
Hosted gate URL	.../WebResources/cr2bf_dactest/index.html access password: dacdemo
Model-driven app	ExecutiveDashboard_test
Solution	CLCPA DAC Dashboard
Source of truth	Coned/CLCPA/ExecutiveDashboard_Test/ (GitHub)

5.2 Dataverse Tables

Display name	Logical name	Role in the flow
DAC Ingest Test - Table Data	cr2bf_dacingesttesttabledata1	One record per table-year; stores the grid as JSON.
DAC Ingest Test Change History	cr2bf_dacingesttestchangehistory	Audit log: WHO / WHEN / what changed.
DAC Ingest Test Reporting Year	cr2bf_dacingesttestreportingyear	The set of available reporting years.

For the full field-level data model behind these flows, see the companion **Data Schema** document. For step-by-step usage and maintenance, see the **Technical / User Documentation**.